



Tenbin Audit Report

Tenbin Audit Report

The Tenbin team engaged Fuzzland to conduct a thorough security audit of their contracts. The primary objective was to identify and mitigate potential security vulnerabilities, risks, and coding issues to enhance the project's robustness and reliability. Fuzzland conducted this assessment over 10 person-days, involving one engineer from Fuzzland and one engineer from Taichi Audit Group who reviewed the code over a span of 5 days. Employing a multifaceted approach that included static analysis, fuzz testing, formal verification, and manual code review, the team has identified 12 issues.

Scope

Project Name	Tenbin
Language / Framework	Solidity (Foundry)
GitHub Repo	 tenbin-contracts 
Commit	<code>820e700943c2b657c54219ab3c99039df188440a</code>
Fix Commit	<code>23a4081ba70e6dd34399781dfefe699004cc4b9</code>
Scope	<code>src/**/*.sol</code>

Disclaimer

The audit does not ensure that it has identified every security issue in the smart contracts, and it should not be seen as a confirmation that there are no more vulnerabilities. The audit is not exhaustive, and we recommend further independent audits and setting up a public bug bounty program for enhanced security verification of the smart contracts. Additionally, this report should not be interpreted as personal financial advice or recommendations.

Auditing Process

- **Static Analysis:** We perform static analysis using our internal tools and Slither to identify potential vulnerabilities and coding issues.
- **Fuzz Testing:** We execute fuzz testing with our internal fuzzers to uncover potential bugs and logic flaws.

- **Invariant Development:** We convert the project into Foundry project and develop Foundry invariant tests for the project based on the code semantics and documentation.
- **Invariant Testing:** We run multiple fuzz testing tools, including Foundry and ItyFuzz, to identify violations of invariants we developed.
- **Formal Verification:** We develop individual tests for critical functions and leverage Halmos to prove the functions in question are not vulnerable.
- **Manual Code Review:** Our engineers manually review code to identify potential vulnerabilities not captured by previous methods.

Vulnerability Severity

We divide severity into three distinct levels: high, medium, low. This classification helps prioritize the issues identified during the audit based on their potential impact and urgency.

- **High (H) Severity Issues** represent critical vulnerabilities or flaws that pose a significant risk to the system's security, functionality, or performance. These issues can lead to severe consequences such as fund loss, or major service disruptions if not addressed immediately. High severity issues typically require urgent attention and prompt remediation to mitigate potential damage and ensure the system's integrity and reliability.
- **Medium (M) Severity Issues** are significant but not critical vulnerabilities or flaws that can impact the system's security, functionality, or performance. These issues might not pose an immediate threat but have the potential to cause considerable harm if left unaddressed over time. Addressing medium severity issues is important to maintain the overall health and efficiency of the system, though they do not require the same level of urgency as high severity issues.
- **Low (L) Severity Issues** are minor vulnerabilities or flaws that have a limited impact on the system's security, functionality, or performance. These issues generally do not pose a significant risk and can be addressed in the regular maintenance cycle. While low severity issues are not critical, resolving them can help improve the system's overall quality and user experience by preventing the accumulation of minor problems over time.

Below is a summary of the vulnerabilities with their current status, highlighting the number of issues identified in each severity category and their resolution progress.

	Number	Resolved
High Severity Issues	0	0
Medium Severity Issues	4	4
Low Severity Issues	8	8

Findings

[M] Controller.verifyOrder() double-counts the staked asset share price during staked redemptions

Summary

`Controller.verifyOrder()` incorrectly applies the staked asset share price when validating redemption orders that use `stakedAsset`. In the live redemption flow, `order.asset_amount` already represents an amount of underlying assets to be withdrawn via `instantUnstake()`. However, `verifyOrder()` treats that amount as though it were a number of shares and multiplies it by `convertToAssets(1e18)` again.

This is a separate issue from any instant unstake fee mismatch. Even if the fee is zero, the oracle check still misprices the order because the denominator used in `verifyOrder()` is inflated by an extra share-price conversion.

Technical Details

The staked redemption execution path shows that `order.asset_amount` is already handled as an amount of underlying assets, not as a quantity of staking shares. In `tenbin-contracts/src/Controller.sol:441-444`, the controller calls `IStakedAsset(stakedAsset).instantUnstake(order.asset_amount, order.payer, order.payer)`. The `IStakedAsset` interface in `tenbin-contracts/src/interface/IStakedAsset.sol:151-161` defines the first parameter of `instantUnstake()` as `assets`, and the function itself returns the number of shares redeemed internally. The conversion from assets to shares therefore happens inside `StakedAsset`, not in the controller.

Despite that, `verifyOrder()` in `tenbin-contracts/src/Controller.sol:531-537` performs an additional conversion in the oracle branch:

```
uint256 sharePrice = IERC4626(stakedAsset).convertToAssets(1e18); assetTokenAmount = Math.mulDiv(order.asset_amount, sharePrice, 1e18);
```

This logic is dimensionally incorrect for the staked redemption path. If `order.asset_amount` is already denominated in assets, multiplying it by `sharePrice` transforms it into a larger synthetic value that no longer matches the economic quantity used by `redeem()`. As a result, the oracle check computes:

```
price = collateralAmount / (assetAmount * sharePrice)
```

when the intended comparison is:

```
price = collateralAmount / assetAmount
```

The error is dormant only while the staked token exchange rate remains exactly 1:1. Once rewards accrue and `convertToAssets(1e18)` rises above `1e18`, the denominator used by `verifyOrder()` becomes too large, causing the implied order price to appear systematically too low versus the oracle reference. For a fairly priced order, the structural delta introduced by this bug is approximately `1 - 1 / sharePrice`. For example, a share price of `1.10e18` produces an artificial deviation of roughly 9.09% before any real market drift or fee effects are considered.

Impact

As staking rewards accumulate and the staked token share price increases above par, the controller becomes progressively more likely to reject valid staked redemption orders with `ExceedsOracleDeltaTolerance`. The failure does not require an attacker to manipulate market prices or exploit unusual state transitions; it follows directly from normal protocol operation once the staking exchange rate appreciates.

In practice, this means the oracle backstop for `stakedAsset` redemptions degrades over time instead of remaining stable. Depending on the configured oracle tolerance, the staked redemption route may first become fragile and later become fully unusable, even when orders are signed at fair market prices. Because this behavior is deterministic and scales with protocol reward accrual, it directly undermines the reliability of the redemption path for staked holders.

Recommendation

`verifyOrder()` should stop converting `order.asset_amount` by the staked asset share price in the `stakedAsset` branch. The oracle denominator should reflect the same economic quantity that `redeem()` actually passes into `instantUnstake()`, which is already an amount of underlying assets. In other words, the share-price adjustment should be removed rather than applied a second time.

Status: Resolved

[M] Oracle price check for staked redemptions does not account for instant unstake fee, consuming tolerance budget and potentially blocking redemptions

Summary

The oracle validation for staked redemptions does not account for the instant unstake fee deducted during execution. As a result, the controller compares collateral against the gross staked asset amount even though the user only receives value against the net amount after fees.

Even when the signed order is economically correct, the controller can still measure it as underpriced relative to the oracle because part of the asset amount is diverted to fees before the redemption is completed. This consumes oracle tolerance for every staked redemption and can convert a valid order into a revert.

Technical Details

In `tenbin-contracts/src/Controller.sol:439-448`, the `redeem()` flow calls `IStakedAsset.instantUnstake(order.asset_amount, ...)` when the order uses the staked asset token. The unstake logic in `tenbin-contracts/src/StakedAsset.sol:239-258` deducts `instantUnstakeFee`, sends that fee to `feeRecipient`, and only delivers the remaining underlying amount to the user-side flow.

However, the oracle validation in `tenbin-contracts/src/Controller.sol:519-546` computes its implied price using the pre-fee asset quantity. In the staked branch, the controller derives `assetTokenAmount` from `order.asset_amount` and share price, but it does not subtract the instant unstake fee before comparing the order price to the oracle reference. This creates a structural gap between the amount the signer prices and the amount the controller validates.

If the signer prices redemption orders against the user's actual net-of-fee proceeds, the on-chain implied price is systematically lower than the oracle by roughly the fee ratio. That gap exists even when the order is otherwise perfectly priced. The oracle tolerance therefore starts partially consumed before any real market deviation is considered.

Impact

If the configured instant unstake fee is equal to or greater than the configured oracle tolerance, staked redemptions can fail systematically with `ExceedsOracleDeltaTolerance`. In that case, the protocol's staked redemption route is effectively unavailable until parameters are changed.

Even when the fee is lower than the tolerance, the remaining protection margin is narrower than operators may expect. A portion of the tolerance budget is always spent on fee mismatch rather than on genuine oracle-vs-order divergence, weakening the intended backstop and making the configured tolerance less meaningful for the staked redemption path.

Recommendation

The oracle formula and redemption economics should use the same denominator. The cleanest remediation is to compute the staked redemption amount on a net-of-fee basis inside `verifyOrder()`, so the controller validates the same economic value the user actually receives and the signer is expected to price.

Status: Resolved

[M] Oracle backstop assumes 1:1 collateral-to-USD and asset-to-XAU pegs, leaving the system unable to reflect true value during depegs

Summary

The current oracle validation in `Controller.verifyOrder()` assumes two economic relationships that are not enforced on-chain: each supported collateral token is treated as if it were worth exactly 1 USD, and each unit of the protocol asset is treated as if it were worth exactly 1 troy ounce of gold. The price check can become unreliable during realistic market stress or future collateral expansion, even when the signed order is economically fair.

Under normal conditions, this simplification may be acceptable if both pegs remain stable within the configured tolerance. However, if the collateral token trades away from its USD peg, or if the protocol asset drifts from its intended gold reference, the controller's oracle backstop no longer measures the real value being exchanged. As a result, the backstop may reject valid orders or fail to identify mispriced ones.

Technical Details

In `tenbin-contracts/src/Controller.sol:519-546`, the contract derives an implied order price by dividing normalized collateral amount by asset amount and compares that value to a single oracle feed returned by `GoldOracleAdapter.getPrice()`. That adapter, defined in `tenbin-contracts/src/oracle/GoldOracleAdapter.sol`, returns only an XAU/USD price. There is no independent oracle leg for the collateral token and no oracle leg confirming that the protocol asset itself continues to track gold.

This design only works if the units on both sides happen to align with the oracle feed. That requires the collateral token to behave as a USD-equivalent unit and the protocol asset to behave as an XAU-equivalent unit. The controller does not enforce either assumption.

`setIsCollateral()` in `tenbin-contracts/src/Controller.sol:229-238` accepts any ERC20 with 6 to 18 decimals, and `setOracleAdapter()` in `tenbin-contracts/src/Controller.sol:273-276` installs a single adapter for all supported collateral types rather than a per-collateral price source.

If a collateral stablecoin trades below par, a correctly priced order will still appear off-market to the controller because the on-chain formula continues to benchmark it against a full USD value. Conversely, if the protocol asset itself diverges from its intended gold reference, the current check may continue to pass even though the asset is no longer fairly priced relative to its real market value. The same structural mismatch would also apply if the team later supports a non-USD collateral token or any collateral whose unit value naturally drifts over time.

Impact

The practical impact is that the oracle backstop can fail in both directions. First, during a collateral depeg, orders that are correctly priced by the off-chain signer against actual market value may revert because the on-chain check interprets them as too far away from the XAU/USD reference. This can impair minting and redemption flows during precisely the conditions when users most need a working exit or entry path.

Recommendation

The oracle check should be aligned with the actual pair being exchanged instead of relying on implied peg relationships. The preferred remediation is to price both legs explicitly: a collateral-to-USD oracle for each supported collateral token, combined with a dedicated oracle that measures the protocol asset against its target reference. The controller can then compare the signed order price against a reference built from those components, rather than against a standalone XAU/USD feed.

Status: Resolved

[M] Shared `blockNumber` causes stale mint and redeem counters to leak across operations in the same block

Summary

`Controller` tracks per-block mint and redeem limits using two separate counters, `blockMints` and `blockRedeems`, but only one shared `blockNumber`. This design causes the counter for one operation type to retain its value from the previous block if the other operation type is the first one executed in the new block. User can hit `ExceedsBlockMintLimit` or `ExceedsBlockRedeemLimit` even though the current block has not actually consumed that side's configured quota.

Technical Details

In `tenbin-contracts/src/Controller.sol:142-155`, the controller stores separate counters for minted and redeemed asset volume:

- `blockMints`
- `blockRedeems`

but only one shared block marker:

- `blockNumber`

The mint path in `tenbin-contracts/src/Controller.sol:335-349` resets `blockMints` only when `block.number > blockNumber`. Likewise, the redeem path in `tenbin-contracts/src/Controller.sol:409-422` resets `blockRedeems` only when `block.number > blockNumber`. In both cases, the function also writes `blockNumber = block.number` when it believes it is seeing the first tracked operation of a new block.

This creates cross-talk between the two limit systems. For example, assume block `N` ends with `blockMints = 90` and `blockNumber = N`, while `blockMintLimit = 100`. In block `N + 1`, if the first tracked action is a redeem, `redeem()` will set `blockRedeems` for the new block and update `blockNumber = N + 1`, but it will not reset `blockMints`. If a mint is then processed later in the same block, `mint()` no longer sees a new block and therefore adds the new mint amount onto the stale `blockMints = 90` value from block `N`. A mint of `15` will be evaluated as `90 + 15 = 105` and revert, even though actual mint volume in block `N + 1` is only `15`.

The same failure occurs in the opposite direction if a mint is the first tracked action of the block and a later redeem inherits a stale `blockRedeems` value from the previous block. The root cause is that block freshness is shared, while the counters themselves are not.

Impact

In blocks where both mints and redeems occur, the operation that executes second may inherit a stale counter from the previous block and fail unexpectedly. This can create intermittent and difficult-to-predict availability issues for users, keepers, or backend relayers, especially during periods of clustered order flow.

Because block ordering is externally influenced, the issue can also be used for limited griefing. A participant who can get a small valid order of one type included first in a block may increase the chance that later orders of the opposite type revert due to stale accounting.

Recommendation

Mint and redeem limits should maintain independent block-tracking state. The simplest fix is to replace the shared `blockNumber` with two separate markers, such as `mintBlockNumber` and `redeemBlockNumber`, so each counter is reset based only on activity of its own type.

An alternative is to keep one shared block marker but reset both `blockMints` and `blockRedeems` together whenever a new block is first observed. Either approach would eliminate stale carry-over. The more robust design is separate block markers, because it preserves the independence already implied by having separate mint and redeem quotas.

Status: Resolved

[L] Curated mint reverts when custodian ratio is set to 100 percent, blocking the curated path until configuration is reconciled

Summary

The curated mint path can revert when the controller's `ratio` is configured to `1e18` (100% of collateral routed to the custodian) while the signed context still marks the order as curated. The failure occurs because the manager-side allocation becomes zero when the ratio is set to 100%, but the controller still attempts to perform a curated vault deposit using that zero amount. The deposit path then fails due to a minimum-shares check that requires a strictly positive result.

Technical Details

In `tenbin-contracts/src/Controller.sol:360-376`, `mint()` calculates the custodian share as `collateral_amount * ratio / 1e18` and transfers the remaining amount to the manager. When `ratio == 1e18`, the manager receives exactly zero collateral. Despite that, the controller still enters the curated branch whenever `context.is_curated` is true and calls `ICollateralManager.deposit()` using the manager-side amount, which is now zero.

`verifyContext()` in `tenbin-contracts/src/Controller.sol:551-557` enforces that `share_price > 0` whenever `is_curated` is enabled. In turn, `CollateralManager.deposit()` in `tenbin-contracts/src/CollateralManager.sol:301-315` deposits the amount into the vault and reverts if the shares received are below the provided minimum. A zero-amount deposit produces zero shares, which is always lower than a strictly positive `share_price`, so the transaction reverts.

This creates a reachable mismatch between two independent control planes. On-chain administrators can set `ratio` to 100%, while the off-chain signer can continue to approve curated contexts. The contract does not reconcile those settings before attempting the vault deposit.

Impact

When this configuration combination occurs, every curated mint reverts until either the ratio is lowered below 100% or the signer stops issuing curated contexts. Previously signed curated orders may also become unexecutable if the ratio changes before they are submitted on-chain.

Recommendation

The controller should not attempt a curated vault deposit when the manager-side amount is zero. A simple contract-side guard in `mint()` that skips `ICollateralManager.deposit()` when the computed manager allocation is zero would eliminate the revert condition and remove the race between on-chain ratio changes and in-flight signed contexts.

Status: Resolved

[L] Payers cannot revoke delegation to de-whitelisted signers – stale delegations auto-reactivate on signer re-addition

Summary

`setDelegateStatus()` requires `signers[signer]` to be true before modifying delegation state (line 207). When a signer is removed, payers cannot revoke their delegation – `setDelegateStatus(signer, false)` reverts with `InvalidSigner`. The delegation persists in storage. If the signer is later re-added, all stale delegations automatically reactivate without payer consent.

Technical Details

- `src/Controller.sol:206-210`: `setDelegateStatus`: line 207 requires `signers[signer]`. This gates BOTH delegation AND revocation on signer status. When `signers[signer]` is false, payer cannot revoke existing delegation.
- `src/Controller.sol:187-191`: `setSignerStatus(false)` does NOT clear `delegates[*][account]`. All delegation state persists, dormant until signer re-addition.
- `src/Controller.sol:495-498`: `verifyOrder` checks `signers[signer]` and `delegates[payer][signer]`. After re-addition, both pass for stale delegations.

A representative scenario is:

1. Payer P delegates to signer S. S is later removed (key compromise).
2. P calls `setDelegateStatus(S, false)` → REVERTS `InvalidSigner`.
3. S re-added by `SIGNER_MANAGER`. `delegates[P][S]` still true.
4. S can immediately sign orders on P's behalf without re-consent.

Impact

When a signer is removed for cause (key compromise), payers who delegated to that signer cannot revoke their delegation. If the signer is re-added, all stale delegations become active without payer consent. The re-added signer can immediately execute orders on behalf of previously-delegating payers.

Recommendation

Decouple revocation from signer status: only gate new delegations (status=true) on signers[signer], allow revocation regardless.

Status: Resolved

[L] removeCollateral does not reset pendingRevenue, allowing phantom revenue extraction if collateral is re-added

Summary

`CollateralManager.removeCollateral()` resets `lastTotalAssets` and deletes the vault mapping but does NOT reset `pendingRevenue`. The NatSpec states 'This function gives up any pending revenue' but the implementation does not clear `pendingRevenue`. If the same collateral is re-added via `addCollateral()` (which also does not clear `pendingRevenue`), the stale value carries over and is treated as legitimate revenue from the new vault.

Technical Details

- `src/CollateralManager.sol:157-170`: `removeCollateral` sets `lastTotalAssets=0`, deletes vault, but never touches `pendingRevenue`. The NatSpec on line 159 says 'gives up any pending revenue' – implementation violates documented intent.
- `src/CollateralManager.sol:140-155`: `addCollateral` initializes `lastTotalAssets` but does NOT check or clear `pendingRevenue`. Stale value from removed vault is inherited.
- `src/CollateralManager.sol:423-436`: `_getRevenue` uses `pendingRevenue` as baseline. With a freshly added vault, `revenue = stale pendingRevenue + any new gains`. The stale portion is phantom.

A representative scenario is:

1. USDC vault earning yield. `pendingRevenue=5000e6`.
2. `removeCollateral(USDC)`: `lastTotalAssets=0`, vault deleted, `pendingRevenue=5000e6` (NOT reset).
3. `addCollateral(USDC, newVault)`: `lastTotalAssets` set to new vault, `pendingRevenue` still `5000e6`.
4. `RevenueModule` collects `5000e6` – extracts phantom revenue from principal.

Impact

In vault migration scenarios (remove + re-add), stale pendingRevenue inflates revenue calculations. RevenueModule can extract phantom revenue, draining principal. Impact bounded by stale pendingRevenue value. Requires the specific admin action sequence of remove then re-add.

Recommendation

Add 'pendingRevenue[collateral] = 0' in removeCollateral, consistent with the documented intent.

Status: Resolved

[L] CollateralManager emergency pause does not fully stop controller-driven collateral flows

Summary

`CollateralManager`'s emergency pause only blocks the manager's own entrypoints. It does not prevent `Controller` from continuing to use the manager's standing ERC20 approvals for non-curated mint and redeem flows. As a result, when `CollateralManager` is placed into `FMLPause`, the system can still mint fresh liabilities against collateral sent into the manager, and it can still redeem out any liquid collateral already sitting in the manager contract balance.

Technical Details

`CollateralManager` defines `FMLPause` through the `notPaused` modifier in `tenbin-contracts/src/CollateralManager.sol:110-114`. This modifier is applied to the manager's own operational functions, including `deposit()`, `withdraw()`, `swap()`, `withdrawRevenue()`, `convertRevenue()`, and `rebalance()` in `tenbin-contracts/src/CollateralManager.sol:301-369`. Once the manager is paused, those entrypoints revert as intended.

However, the manager separately grants the controller a standing allowance over supported collateral in `addCollateral()` at `tenbin-contracts/src/CollateralManager.sol:150`, using:

```
IERC20(collateral).forceApprove(controller, type(uint256).max);
```

That allowance remains active during `FMLPause`. In the controller's non-curated mint flow, `Controller.mint()` transfers collateral directly from the payer into the manager at `tenbin-contracts/src/Controller.sol:368-370`. If `context.is_curated` is false, the controller does not call `CollateralManager.deposit()`, so no paused manager entrypoint is touched before the asset liability is minted.

Likewise, in the redeem flow, `Controller.redeem()` only calls `CollateralManager.withdraw()` when `context.is_curated` is true, at `tenbin-contracts/src/Controller.sol:431-434`. But regardless of curation mode, it then attempts to transfer collateral from the manager to the recipient using the manager's standing allowance at `tenbin-contracts/src/Controller.sol:437`. This means that any collateral already sitting idle in the manager contract can still be redeemed out during manager pause, even though the manager itself is in emergency mode. The pause therefore blocks vault-mediated withdrawals, swaps, and rebalancing, but it does not isolate the manager from controller-driven balance movements that use pre-approved token allowances.

Impact

On emergency operations and incident response. During a manager pause, operators may reasonably expect the manager-side collateral surface to be frozen. In practice, that is not fully true: non-curated mints can continue to route new collateral into the manager while minting fresh liabilities, and non-curated redeems can continue to consume whatever liquid collateral is already held in the manager contract.

Recommendation

If the intended semantics of `CollateralManager.FMLPause` are to fully freeze manager-related collateral flows, the controller and manager should be tightened so that manager pause also blocks controller paths that depend on manager balances or allowances. One practical approach is for the controller to query the manager's pause state and reject mint and redeem paths that rely on manager collateral whenever the manager is in `FMLPause`.

Status: Resolved

[L] setFeeRecipient validation logic is inverted, allowing feeRecipient to be zeroed when fee is active

Summary

setFeeRecipient() validation prevents setting feeRecipient to address(0) when instantUnstakeFee == 0 (harmless case), but allows setting it to address(0) when instantUnstakeFee > 0. With feeRecipient=address(0) and non-zero fee, instantUnstake silently skips all fee collection.

Technical Details

- `src/StakedAsset.sol:322-327`: Check 'if (instantUnstakeFee == 0 && newFeeRecipient == address(0)) revert' blocks the harmless case. When fee > 0, recipient can be set to address(0) without revert.
- `src/StakedAsset.sol:248-253`: Fee guard: 'if (instantUnstakeFee > 0 && feeRecipient != address(0))'. Setting feeRecipient to address(0) with fee > 0 causes fees to be silently skipped.

A representative scenario is:

1. instantUnstakeFee=5%. Admin calls setFeeRecipient(address(0)) – succeeds.
2. User instant-unstakes 1000 → fee=0 (skipped because feeRecipient==address(0)), user gets full 1000.

Impact

If DEFAULT_ADMIN_ROLE (5/7 multisig) sets feeRecipient to address(0) while fee > 0, instant unstaking fees are silently skipped. Protocol fee revenue lost. Risk mitigated by high trust threshold for admin role.

Recommendation

Invert the condition: 'if (instantUnstakeFee > 0 && newFeeRecipient == address(0)) revert'.

Status: Resolved

[L] cooldownShares returns share count instead of asset amount due to explicit return overriding named return variable

Summary

StakedAsset.cooldownShares declares returns (uint256 assets, uint256 id) and correctly computes the asset amount internally (line 178: `assets = previewRedeem(shares)`), but an explicit return (`shares, id`) on line 188 overrides the named return variable. The function returns the input share count in the position where callers expect the converted asset amount. Internal cooldown state and events are correct – only the return value to callers is wrong.

Technical Details

- `src/StakedAsset.sol:173-189`: Named return variable 'assets' is correctly assigned via `previewRedeem(shares)` at line 178 and stored in cooldown at line 183. But line 188 has `'return (shares, id)'` which overrides the named return, returning shares in the assets position.

A representative scenario is:

Given `sharePrice=1.5`, user calls `cooldownShares(100e18)`:

- `previewRedeem(100e18) = 150e18 assets`. cooldown stores 150e18 (correct).
- Event emits `assets=150e18` (correct).
- Return value: `(100e18, id)` – returns shares (100) instead of assets (150).

Impact

Any contract or off-chain integrator reading the first return value of `cooldownShares()` receives the share count instead of the asset amount. Since `shares < assets` when `sharePrice > 1`, callers underestimate the cooldown value. No in-scope Tenbin contract currently reads this return value, but the `IStakedAsset` interface documents it as returning assets, creating correctness risk for current or future integrations.

Recommendation

Change line 188 from 'return (shares, id)' to 'return (assets, id)',
or remove the explicit return statement.

Status: Resolved

[L] SwapModule increases allowance to the 1inch router without resetting, allowing leftover allowance to persist across swaps

Summary

`SwapModule.swap1Inch()` increases token allowance for the 1inch router before each swap, but it does not clear that allowance after execution. In the currently audited integration, this does not appear to create an immediate exploit because the router address is fixed and the surrounding flow expects the approved amount to be spent.

Technical Details

In `tenbin-contracts/src/SwapModule.sol:84-92`, the module calls `safeIncreaseAllowance(params.router, params.amount)` immediately before invoking the 1inch router. There is no corresponding approval reset after the swap returns. Because `safeIncreaseAllowance` is additive, any pre-existing allowance is carried forward and increased rather than overwritten.

The current implementation partially mitigates this through the `spentAmount < params.amount` check, which is intended to reject under-spent swaps. However, the module still relies on the router and execution path to consume allowance exactly as expected. The approval state is not normalized by the module before or after the external call.

This matters because the module is granting spend rights to an external contract. If router behavior changes, if a future integration path leaves residue, or if a residual allowance exists for any other reason, that allowance remains available until explicitly revoked. The issue is therefore less about present swap correctness and more about durable control over token approvals.

Impact

Under the audited assumptions, immediate impact is limited because the current router integration is expected to consume the approved amount in normal operation. Nevertheless, any leftover allowance would persist across transactions and could accumulate over time as curator swaps continue.

If the external router were ever compromised, upgraded in a harmful way outside Tenbin's control, or interacted with through an unexpected execution path, any residual allowance would become directly usable to pull tokens from the module while it temporarily holds swap inventory. The exposure is therefore bounded, but it is avoidable and unnecessary.

Recommendation

The module should grant an exact per-call allowance and then clear it immediately after the swap. Using `forceApprove` to set allowance to the required amount before execution and back to zero afterward is the standard hardening pattern for router integrations, including tokens that require zero-first approval semantics.

Status: Resolved

[L] Controller.multicall uses unpermissioned delegatecall-to-self, safe today but fragile under any future payable function

Summary

`Controller.multicall()` currently lets any caller batch arbitrary calls to the controller through `delegatecall` back into the same contract. In the audited codebase, this does not create an immediate exploit because the controller does not currently expose payable entry points and privileged functions still re-check `msg.sender`.

Technical Details

In `tenbin-contracts/src/Controller.sol:308-317`, `multicall()` iterates over an array of calldata blobs and executes each one using `address(this).delegatecall(data[i])`. Because `delegatecall` preserves execution context, each subcall sees the original `msg.sender`, `msg.value`, and controller storage.

At present, that preserved context does not appear to break authorization. Administrative functions continue to enforce role checks against the original caller, and public user-facing functions such as `mint()` and `redeem()` perform their own signature and nonce validation. The controller also does not currently contain a payable function that consumes `msg.value`.

However, the pattern becomes fragile if the controller later adds a payable feature such as native-token fee collection, an ETH-based deposit path, or any helper function that uses `msg.value`. In that case, a single multicall transaction could replay the same `msg.value` across multiple internal subcalls. This is a well-known failure mode in delegatecall-based multicall implementations and has led to real production incidents in other systems.

Impact

There is no confirmed exploit in the present deployment logic, so this issue does not create immediate fund loss today. If that promise is overlooked during a later upgrade, refactor, or product fork, a single newly introduced payable function could allow a caller to obtain multiple credits or state updates while only providing one payment. Even outside of `msg.value` handling, multicall also allows public functions to be sequenced atomically in ways that may invalidate future assumptions about transaction ordering.

Recommendation

No action needed.

Status: Resolved